The Linux Command Line
Une introduction complète (2e édition)

par William Shotts (No Starch Press)

Résumé complet du livre – avec des exemples concrets

Aperçu chapitre par chapitre avec commandes/outils, points à retenir et un exemple pratique pour chacun

Préparé comme compagnon de lecture – et non comme substitut au texte intégral.

À propos de ce résumé

Ce document résume les 36 chapitres de The Linux Command Line, 2e édition de William Shotts. Cette version révisée ajoute un exemple exécutable sous chaque concept clé, de sorte que chaque puce est associée à une petite illustration concrète de ligne de commande plutôt qu'à une simple description.

Chaque section de chapitre comprend : un synopsis, des concepts clés (chacun suivi d'un exemple encadré ombré montrant la syntaxe/sortie réelle de la commande) et un encadré répertoriant chaque commande/outil couvert. Les commandes apparaissant pour la première fois dans le livre sont signalées en orange.

Structure du livre (4 parties, 36 chapitres)

- Partie I — Apprentissage du Shell (Ch. 1-10) : bases du terminal, navigation, exploration du système, manipulation de fichiers, commandes, redirection, expansion/citation, astuces clavier, autorisations, processus.
- Partie II — Configuration et environnement (Ch. 11-13) : variables d'environnement, l'éditeur vi et personnalisation de l'invite du shell.
- Partie III — Tâches communes et outils essentiels (Ch. 14-23) : gestion des packages, supports de stockage, mise en réseau, recherche, archivage/sauvegarde, expressions régulières, traitement de texte, formatage de sortie, impression et compilation de programmes.
- Partie IV — Écriture de scripts Shell (Ch. 24 à 36) : depuis un premier script jusqu'aux variables, fonctions, toutes les constructions de contrôle de flux (if/case/while/for), gestion des entrées, dépannage, chaînes/nombres/tableaux et fonctionnalités avancées du shell ("exotica").

Partie I - Apprendre le Shell

Chapitre 1 : Qu'est-ce que la coque ?

Présente le shell en tant qu'interface de commande textuelle pour Linux, accessible via un émulateur de terminal. Couvre le lancement d'un terminal, les premières frappes au clavier (historique des commandes, mouvement du curseur), l'essai de quelques commandes simples (date, cal, df) et la fin correcte d'une session de terminal.

Concepts et exemples clés

- Le shell est un programme qui lit les commandes saisies et demande au noyau de les exécuter.

```
$ date
Sat Oct 24 10:15:02 UTC 2026
```

- La plupart des distributions Linux utilisent par défaut bash (Bourne Again SHell).

```
$ echo $SHELL
/bin/bash
```

- Une session de terminal peut être terminée avec la commande exit ou en fermant la fenêtre du terminal.

```
$ exit
```

Commandes / outils couverts dans ce chapitre

`date`	`cal`	`df`	`exit`

★ Nouveaux dans ce chapitre : date, cal, df

Chapitre 2 : Navigation

Couvre la structure arborescente du système de fichiers Linux, le concept du répertoire de travail actuel et comment s'y déplacer. Présente des chemins absolus et relatifs et des raccourcis pratiques (~, ., .., -) pour une navigation plus rapide.

Concepts et exemples clés

- Le système de fichiers est une seule arborescence unifiée dont la racine est /, contrairement aux systèmes à lettre de lecteur.

```
$ ls /
bin boot dev etc home usr var ...
```

- pwd affiche le répertoire actuel ; cd le change ; les chemins absolus commencent à /, les chemins relatifs commencent à l'emplacement actuel.

```
$ pwd
/home/alice
$ cd /etc
$ cd Documents
```

- Raccourcis : ~ (accueil), . (répertoire actuel), .. (répertoire parent), - (répertoire précédent) accélère la navigation.

```
$ cd ~
$ cd ..
$ cd -
```

Commandes / outils couverts dans ce chapitre

`pwd`	`cd`

★ Nouveaux dans ce chapitre : pwd, cd

Chapitre 3 : Explorer le système

Approfondit la liste des répertoires avec les options ls et la sortie au format long (autorisations, propriétaire, taille, horodatages), en déterminant les types de fichiers avec file et en affichant le contenu des fichiers avec less. Fait une visite guidée des répertoires système clés, puis explique les liens symboliques et physiques.

Concepts et exemples clés

- Le format long de ls -l révèle le type de fichier, les autorisations, le nombre de liens, le propriétaire, le groupe, la taille et l'heure de modification dans une seule vue.

```
$ ls -l /etc/hosts
-rw-r--r-- 1 root root 220 Jan 10 09:00 /etc/hosts
```

- less est le téléavertisseur standard pour visualiser (et non éditer) le contenu des fichiers, prenant en charge la recherche.

```
$ less /etc/passwd
(press / then type a word to search, q to quit)
```

- Les liens symboliques sont des pointeurs vers un chemin ; les liens physiques sont des noms supplémentaires pour le même inode.

```
$ ln -s /etc/hosts myhosts
$ ls -l myhosts
lrwxrwxrwx 1 alice alice 10 ... myhosts -> /etc/hosts
```

Commandes / outils couverts dans ce chapitre

`ls`	`file`	`less`

★ Nouveaux dans ce chapitre : ls, file, less

Chapitre 4 : Manipulation de fichiers et de répertoires

Couvre les caractères génériques (modèles de globalisation) pour faire correspondre plusieurs noms de fichiers, puis les commandes principales de gestion de fichiers : mkdir, cp, mv, rm et ln. Un exercice pratique de « terrain de jeu » permet au lecteur de s'entraîner à créer, copier, déplacer, lier et supprimer des fichiers/répertoires.

Concepts et exemples clés

- Les caractères génériques (*, ?, [...]) permettent à une seule commande d'opérer sur plusieurs fichiers correspondants à la fois.

```
$ ls *.txt
$ rm file?.log
```

- cp/mv/rm prennent tous en charge les options récursives (-r) et interactives/forces pour la sécurité et les opérations en vrac.

```
$ cp -r project/ project_backup/
$ rm -i old_file.txt
```

- ln crée des liens physiques par défaut, ou des liens symboliques avec -s.

```
$ ln original.txt hardlink.txt
$ ln -s original.txt softlink.txt
```

Commandes / outils couverts dans ce chapitre

`mkdir`	`cp`	`mv`	`rm`	`ln`

★ Nouveaux dans ce chapitre : mkdir, cp, mv, rm, ln

Chapitre 5 : Travailler avec les commandes

Explique ce que sont réellement les commandes (exécutables, éléments intégrés, fonctions, alias) et comment les identifier avec leur type et lequel. Couvre l'obtention de la documentation via help, --help, man, apropos, whatis et info, ainsi que les fichiers README et la création de commandes personnalisées avec alias.

Concepts et exemples clés

- type révèle comment le shell interprétera un mot : alias, élément intégré, fichier ou fonction.

```
$ type cd
cd is a shell builtin
$ type ls
ls is /usr/bin/ls
```

- l'homme est la référence canonique ; à propos/whatis recherche les résumés des pages de manuel par mot-clé.

```
$ man ls
$ apropos partition
```

- alias permet aux utilisateurs de définir leurs propres commandes abrégées (par exemple, alias ll='ls -l').

```
$ alias ll='ls -l'
$ ll
```

Commandes / outils couverts dans ce chapitre

`type` `whatis`	`which` `info`	`help` `alias`	`man`	`apropos`

★ Nouveaux dans ce chapitre : type, which, man, apropos, whatis, info, alias

---

Chapitre 6 : Redirection

Couvre les flux d'entrée/sortie/erreur standard et les redirige vers/depuis des fichiers (>, >>, 2>, &>), ainsi que cat pour la concaténation/visualisation des fichiers. Présente des pipelines (|) pour enchaîner les commandes et un ensemble d'utilitaires de filtrage classiques : uniq, wc, grep, head/tail et tee.

Concepts et exemples clés

- Chaque processus dispose de trois flux par défaut : stdin (0), stdout (1), stderr (2), tous redirigés indépendamment.

```
$ ls /nonexistent 2> errors.txt
$ ls > out.txt 2>&1
```

- Les pipelines connectent la sortie standard d'une commande directement au stdin de la commande suivante, permettant une composition de commandes puissante.

```
$ ls -l | grep '^d' | wc -l
```

- grep (correspondance de motifs) et la famille de filtres head/tail/wc/uniq sont des outils fondamentaux réutilisés constamment dans le reste du livre.

```
$ sort names.txt | uniq -c
$ tail -n 20 /var/log/syslog
```

Commandes / outils couverts dans ce chapitre

`cat` `tail`	`uniq` `tee`	`wc`	`grep`	`head`

★ Nouveaux dans ce chapitre : cat, uniq, wc, grep, head, tail, tee

---

Chapitre 7 : Voir le monde tel que le voit la shell

Explique les mécanismes d'expansion du shell que le shell exécute avant d'exécuter une commande : expansion du nom de chemin (caractères génériques), expansion des tildes, expansion arithmétique, expansion des accolades, expansion des paramètres et substitution de commande. Couvre également les citations (doubles/simples) et les séquences d'échappement pour contrôler ou empêcher l'expansion.

Concepts et exemples clés

- Comprendre l'ordre d'expansion est essentiel pour prédire ce qu'une commande reçoit réellement comme arguments.

```
$ echo file{1..3}.txt
file1.txt file2.txt file3.txt
```

- La substitution de commande ($(command)) intègre la sortie d'une commande directement dans une autre ligne de commande.

```
$ echo "Today is $(date +%A)"
Today is Saturday
```

- Les guillemets doubles suppriment la plupart des développements, à l'exception du développement des paramètres/commandes/arithmétique ; Les guillemets simples suppriment toute expansion.

```
$ echo "$HOME has $(ls | wc -l) files"
$ echo 'Literal $HOME is not expanded'
```

Commandes / outils couverts dans ce chapitre

```
echo (expansion
demonstration)
```

★ Nouveaux dans ce chapitre : echo

Chapitre 8 : Astuces avancées pour le clavier

Couvre l'édition efficace de la ligne de commande (mouvement du curseur, modification du texte, couper/coller "tuer et tirer"), la complétion des onglets et l'utilisation de l'historique des commandes - y compris l'historique de recherche et l'expansion de l'historique (par exemple, !!, !n) pour réémettre rapidement les commandes passées.

Concepts et exemples clés

- Les raccourcis d'édition basés sur Readline (Ctrl-A/E/U/K/Y, etc.) accélèrent considérablement l'utilisation de la ligne de commande.

```
Ctrl-A (jump to start of line)
Ctrl-E (jump to end of line)
Ctrl-K (cut to end of line)
```

- La complétion par tabulation réduit la saisie et les fautes de frappe pour les noms de fichiers, les commandes et les variables.

```
$ cd Doc<TAB>
-> cd Documents/
```

- L'expansion de l'historique (!!, !$, !string) permet aux utilisateurs de réutiliser ou de référencer des commandes antérieures de manière concise.

```
$ sudo !!
$ cp report.txt !$
```

Commandes / outils couverts dans ce chapitre

```
history
```

★ Nouveaux dans ce chapitre : history

Chapitre 9 : Autorisations

Explique le modèle d'autorisation propriétaire/groupe/autre de Linux et les bits de lecture/écriture/exécution, puis chmod (modes symbolique et octal) pour les modifier, umask pour les autorisations par défaut et les autorisations spéciales (setuid, setgid, sticky bit). Couvre la modification de l'identité d'un utilisateur/groupe avec su et sudo, et la propriété avec chown/chgrp.

Concepts et exemples clés

- Chaque fichier/répertoire dispose de trois ensembles d'autorisations (propriétaire, groupe, autre), chacun avec lecture/écriture/exécution.

```
$ ls -l script.sh
-rwxr-xr-- 1 alice staff 512 ... script.sh
```

- chmod accepte les modes symbolique (u+x) et numérique/octal (755).

```
$ chmod u+x script.sh
$ chmod 755 script.sh
```

- sudo (exécuter en tant qu'autre utilisateur, généralement root) est l'alternative moderne et auditée à su pour les commandes privilégiées.

```
$ sudo apt update
$ sudo chown root:root /etc/myconfig
```

Commandes / outils couverts dans ce chapitre

`chmod` `chgrp`	`umask` `passwd`	`su`	`sudo`	`chown`

★ Nouveaux dans ce chapitre : chmod, umask, su, sudo, chown, chgrp, passwd

---

Chapitre 10 : Processus

Explique comment fonctionnent les processus Linux (relations parent/enfant, PID), en les visualisant de manière statique avec ps et dynamiquement avec top, et en les contrôlant : interruption, mise en arrière-plan/premier plan et arrêt. Couvre les signaux Unix et leur envoi avec kill/killall, ainsi que l'arrêt du système en toute sécurité.

Concepts et exemples clés

- Chaque programme en cours d'exécution est un processus avec un PID unique, visible via ps ou top.

```
$ ps aux | grep firefox
$ top
```

- Ctrl-C envoie SIGINT ; Ctrl-Z suspend un processus (SIGTSTP) ; jobs/bg/fg gère l'état d'arrière-plan/premier plan.

```
$ sleep 100
^Z
[1]+ Stopped sleep 100
$ bg %1
```

- kill envoie des signaux (pas seulement "tuer" le processus) - par exemple, SIGTERM pour une terminaison en douceur, SIGKILL pour le forcer.

```
$ kill 4821
$ kill -9 4821
$ killall firefox
```

Commandes / outils couverts dans ce chapitre

ps	top	jobs	bg	fg
kill	killall	shutdown		

★ Nouveaux dans ce chapitre : ps, top, jobs, bg, fg, kill, killall, shutdown

---

Partie II — Configuration et environnement

Chapitre 11 : L'environnement

Explique ce qui est stocké dans l'environnement shell (variables d'environnement), comment l'environnement est établi lors de la connexion via les fichiers de démarrage (par exemple, .bash_profile, .bashrc) et comment le modifier en toute sécurité à l'aide d'un éditeur de texte pour modifier les fichiers de démarrage et activer les modifications.

Concepts et exemples clés

- Les variables d'environnement (par exemple, PATH, HOME, PS1) configurent le comportement du shell et du programme globalement pour la session.

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin
```

- Les fichiers de démarrage s'exécutent automatiquement lors de la connexion/du démarrage du shell et constituent l'emplacement standard pour conserver les personnalisations de l'environnement.

```
$ nano ~/.bashrc
(add: export EDITOR=vim)
```

- Les modifications apportées aux fichiers de démarrage nécessitent un resourcing (source) ou une nouvelle session pour prendre effet.

```
$ source ~/.bashrc
```

Commandes / outils couverts dans ce chapitre

env / printenv	export	source

★ Nouveaux dans ce chapitre : env/printenv, export, source

Chapitre 12 : Une introduction douce à vi

Un tutoriel pratique sur l'éditeur de texte vi (omniprésent sur presque tous les systèmes de type Unix), couvrant le démarrage/arrêt, les trois modes d'édition (normal, insertion, ligne de commande), le mouvement du curseur, l'édition de base (ajouter, supprimer, couper/copier/coller, joindre des lignes), la recherche et le remplacement et l'édition de plusieurs fichiers.

Concepts et exemples clés

- L'édition modale de vi (modes séparés pour la navigation et la saisie) est le principal obstacle conceptuel pour les nouveaux arrivants, mais permet une édition très rapide une fois apprise.

```
$ vi notes.txt
(press i to enter Insert mode, Esc to return to Normal mode)
```

- Connaître vi de base est précieux car il est disponible par défaut sur pratiquement tous les systèmes Linux/Unix, contrairement aux éditeurs plus complets.

```
:wq (save and quit)
:q! (quit without saving)
```

- La recherche et le remplacement global (:%s/old/new/g) reflètent la syntaxe sed abordée plus loin dans le livre.

```
:%s/foo/bar/g
```

Commandes / outils couverts dans ce chapitre

```
vi / vim
```

★ Nouveaux dans ce chapitre : vi/vim

Chapitre 13 : Personnalisation de l'invite

Couvre l'anatomie de l'invite bash (variable PS1 et ses séquences d'échappement), la création de conceptions d'invite alternatives, l'ajout de couleur avec les codes d'échappement ANSI, le déplacement du curseur dans l'invite et l'enregistrement permanent de l'invite personnalisée dans un fichier de démarrage.

Concepts et exemples clés

- PS1 contrôle la chaîne d'invite principale à l'aide de séquences spéciales d'échappement avec barre oblique inverse (\u, \h, \w, etc.).

```
$ PS1='[\u@\h \W]\$ '
[alice@laptop ~]$
```

- Les codes de couleur ANSI peuvent être intégrés dans PS1 pour distinguer visuellement les éléments d'invite.

```
PS1='\[\e[32m\]\u@\h\[\e[0m\]:\w\$ '
```

- Les personnalisations d'invite doivent être enregistrées dans .bashrc pour persister d'une session à l'autre.

```
$ echo "PS1='[\u@\h \W]\$ '" >> ~/.bashrc
```

Commandes / outils couverts dans ce chapitre

```
PS1 (prompt
variable)
```

Partie III — Tâches courantes et outils essentiels

Chapitre 14 : Gestion des packages

Explique le fonctionnement des systèmes de packaging Linux (fichiers de package, référentiels, dépendances) et la différence entre les outils de haut niveau (par exemple, apt, yum/dnf) et les outils de bas niveau (dpkg, rpm). Parcourez les tâches courantes : recherche, installation, suppression et mise à jour de packages, liste des packages installés et recherche du package propriétaire d'un fichier donné.

Concepts et exemples clés

- Les outils de haut niveau (apt-get/apt, yum/dnf) résolvent automatiquement les dépendances ; les outils de bas niveau (dpkg, rpm) fonctionnent directement sur les fichiers de packages individuels.

```
$ sudo apt install htop
$ sudo dpkg -i package.deb
```

- Les référentiels sont des collections organisées de packages dont le gestionnaire de packages extrait.

```
$ sudo apt update
$ apt search nginx
```

- Des outils existent pour retracer un fichier jusqu'au package qui l'a installé, utiles pour le dépannage.

```
$ dpkg -S /bin/ls
coreutils: /bin/ls
```

Commandes / outils couverts dans ce chapitre

`apt-get / apt`	`dpkg`	`yum / dnf`	`rpm`

★ Nouveaux dans ce chapitre : apt-get/apt, dpkg, yum/dnf, rpm

---

Chapitre 15 : Supports de stockage

Couvre le montage/démontage des périphériques de stockage, l'affichage des systèmes de fichiers montés et la détermination des noms de périphériques, la création de nouveaux systèmes de fichiers avec fdisk (partitionnement) et mkfs, le test/réparation des systèmes de fichiers et le déplacement de données directement vers/depuis les périphériques (création et écriture d'images CD-ROM/ISO).

Concepts et exemples clés

- mount/umount attache et détache les systèmes de fichiers à l'arborescence de répertoires à un point de montage choisi.

```
$ sudo mount /dev/sdb1 /mnt/usb
$ sudo umount /mnt/usb
```

- fdisk manipule les tables de partition ; mkfs crée un nouveau système de fichiers sur une partition.

```
$ sudo fdisk -l
$ sudo mkfs -t ext4 /dev/sdb1
```

- La création/écriture d'images ISO couvre les flux de travail sur supports optiques existants, encore parfois utiles.

```
$ dd if=/dev/sr0 of=disk.iso bs=2048
```

Commandes / outils couverts dans ce chapitre

`mount` `dd`	`umount`	`fdisk`	`mkfs`	`fsck`

★ Nouveaux dans ce chapitre : mount, umount, fdisk, mkfs, fsck, dd

---

Chapitre 16 : Réseautage

Couvre l'examen et la surveillance d'un réseau (ping, traceroute, ip, netstat), le transport de fichiers sur un réseau (ftp, le lftp amélioré et wget pour le téléchargement) et la communication à distance sécurisée avec ssh, ainsi que les scp et sftp associés pour un transfert de fichiers sécurisé.

Concepts et exemples clés

- ping/traceroute diagnostique la connectivité et le routage de base ; ip gère les interfaces réseau ; netstat rapporte les connexions/ports d'écoute.

```
$ ping -c 4 example.com
$ ip addr show
```

- ssh fournit un accès shell distant crypté et constitue l'épine dorsale de l'administration Linux à distance sécurisée.

```
$ ssh alice@192.168.1.10
```

- Transfert de fichiers sécurisé par couche scp et sftp en plus du protocole SSH.

```
$ scp report.pdf alice@server:/home/alice/
$ wget https://example.com/file.tar.gz
```

Commandes / outils couverts dans ce chapitre

ping	traceroute	ip	netstat	ftp
lftp	wget	ssh	scp	sftp

★ Nouveaux dans ce chapitre : ping, traceroute, ip, netstat, ftp, lftp, wget, ssh, scp, sftp

---

Chapitre 17 : Recherche de fichiers

Couvre la localisation pour des recherches rapides dans une base de données de noms de fichiers prédéfinie et la commande find bien plus puissante pour les recherches complexes utilisant des tests, des opérateurs, des actions prédéfinies et des actions définies par l'utilisateur. Présente xargs pour appliquer efficacement des commandes à la sortie de find et discute des considérations en matière de performances.

Concepts et exemples clés

- la localisation est rapide mais s'appuie sur une base de données périodiquement mise à jour ; find recherche dans le système de fichiers actif et prend en charge des critères riches.

```
$ locate httpd.conf
$ find /var/log -name '*.log'
```

- L'action -exec de find (ou la transmission vers xargs) permet aux résultats de la recherche d'alimenter directement une autre commande.

```
$ find . -name '*.tmp' -exec rm {} \;
```

- xargs regroupe efficacement les arguments, évitant ainsi les erreurs de « liste d'arguments trop longue » provenant de boucles naïves.

```
$ find . -name '*.log' | xargs rm
```

Commandes / outils couverts dans ce chapitre

locate	find	xargs

★ Nouveaux dans ce chapitre : locate, find, xargs

Chapitre 18 : Archivage et sauvegarde

Couvre la compression de fichiers avec gzip et bzip2, l'archivage avec tar (la combinaison classique avec compression) et zip, et la synchronisation de fichiers/répertoires avec rsync, y compris l'utilisation de rsync sur un réseau pour des sauvegardes incrémentielles efficaces.

Concepts et exemples clés

- gzip/bzip2 compresse des fichiers uniques ; tar regroupe plusieurs fichiers/répertoires dans une seule archive, généralement compressés ensemble (tar.gz, tar.bz2).

```
$ gzip bigfile.txt
$ tar -czvf backup.tar.gz Documents/
```

- zip est principalement utilisé pour les archives multiplateformes (compatibles Windows).

```
$ zip -r archive.zip Photos/
```

- rsync synchronise efficacement les fichiers en transférant uniquement les parties modifiées, idéal pour les sauvegardes et la mise en miroir à distance.

```
$ rsync -av Documents/ /mnt/backup/Documents/
```

Commandes / outils couverts dans ce chapitre

gzip	bzip2	tar	zip / unzip	rsync

★ Nouveaux dans ce chapitre : gzip, bzip2, tar, zip/unzip, rsync

---

Chapitre 19 : Expressions régulières

Une introduction approfondie aux expressions régulières construite autour de grep : métacaractères et littéraux, le caractère générique "n'importe quel caractère", les ancres, les expressions entre crochets/classes de caractères, les classes de caractères POSIX, les expressions régulières de base et étendues POSIX, l'alternance et les quantificateurs (?, *, +, {}).

Concepts et exemples clés

- Les expressions régulières sont un langage de correspondance de modèles utilisé dans de nombreux outils de texte Unix (grep, sed, find, vim, etc.).

```
$ grep '^root' /etc/passwd
```

- Les expressions régulières POSIX Basic (BRE) et étendues (ERE) diffèrent dans la manière dont les métacaractères doivent être échappés.

```
$ grep -E '(cat|dog)' animals.txt
```

- Les quantificateurs et les expressions entre parenthèses sont les éléments de base permettant de faire correspondre des modèles du monde réel tels que des numéros de téléphone ou des noms de fichiers.

```
$ grep -E '^[0-9]{3}-[0-9]{4}$' phonelist.txt
```

Commandes / outils couverts dans ce chapitre

grep (regex engine)	egrep / grep -E

★ Nouveaux dans ce chapitre : egrep/grep -E

Chapitre 20 : Traitement de texte

Une boîte à outils approfondie pour manipuler du texte : revisiter cat/sort/uniq, découper et découper en dés avec couper, coller et joindre, comparer le texte avec comm, diff et patch, et éditer à la volée avec tr (translittération), sed (l'éditeur de flux, largement couvert) et aspell pour la vérification orthographique.

Concepts et exemples clés

- trier, couper, coller et joindre pour former une boîte à outils puissante pour restructurer les données textuelles tabulaires/lignes.

```
$ cut -d: -f1 /etc/passwd | sort
```

- diff/patch forment ensemble le flux de travail classique pour distribuer et appliquer les modifications du code source.

```
$ diff old.c new.c > fix.patch
$ patch old.c < fix.patch
```

- sed permet des transformations de texte non interactives et scriptables sur des fichiers ou des flux entiers.

```
$ sed 's/foo/bar/g' input.txt > output.txt
```

Commandes / outils couverts dans ce chapitre

`sort` `diff`	`cut` `patch`	`paste` `tr`	`join` `sed`	`comm` `aspell`

★ Nouveaux dans ce chapitre : sort, cut, paste, join, comm, diff, patch, tr, sed, aspell

---

Chapitre 21 : Formatage de la sortie

Couvre les outils de formatage simples : nl (lignes numériques), plier (lignes de retour à la ligne), fmt (formatage de texte simple), pr (format d'impression) et printf (sortie formatée avec précision). Présente les systèmes de formatage de documents, en particulier groff.

Concepts et exemples clés

- printf offre un contrôle précis sur le formatage de la sortie, utile à la fois de manière interactive et très utile dans les scripts shell.

```
$ printf '%-10s %5d\n' Apples 42
```

- pr/fmt/fold prépare le texte brut pour un affichage ou une impression lisible.

```
$ fold -w 60 longline.txt
```

- groff représente la chaîne d'outils classique de formatage/composition de documents Unix, toujours utilisée pour les pages de manuel elles-mêmes.

```
$ groff -man -Tascii mypage.1 | less
```

Commandes / outils couverts dans ce chapitre

`nl` `groff`	`fold`	`fmt`	`pr`	`printf`

★ Nouveaux dans ce chapitre : nl, fold, fmt, pr, printf, groff

---

Chapitre 22 : Impression

Un bref historique de la technologie d'impression, puis l'impression pratique sous Linux : préparer des fichiers avec pr, envoyer des travaux d'impression avec lpr (style Berkeley) ou lp (style System V), surveiller/contrôler les travaux d'impression avec lpstat, lpq et lprm/cancel.

Concepts et exemples clés

- Linux prend en charge deux familles de commandes d'impression historiques (style BSD et style System V), toutes deux toujours présentes via CUPS.

```
$ lpr report.pdf
$ lp report.pdf
```

- pr et a2ps convertissent le texte brut en sortie formatée prête à imprimer.

```
$ pr -h "Monthly Report" data.txt | lpr
```

- Les outils de surveillance permettent aux utilisateurs de vérifier l'état de la file d'attente et d'annuler les tâches d'impression bloquées ou indésirables.

```
$ lpq
$ lprm 42
```

Commandes / outils couverts dans ce chapitre

`pr` `lpq`	`lpr` `lprm`	`lp` `cancel`	`a2ps`	`lpstat`

★ Nouveaux dans ce chapitre : lpr, lp, a2ps, lpstat, lpq, lprm, cancel

---

Chapitre 23 : Compilation de programmes

Explique ce qu'est la compilation, puis décrit la compilation d'un vrai programme C à partir des sources : obtention du code source, examen de l'arborescence des sources, construction du programme (généralement via ./configure && make) et installation (make install).

Concepts et exemples clés

- La compilation traduit le code source lisible par l'homme en binaires exécutables par machine.

```
$ gcc -o hello hello.c
$ ./hello
```

- Le workflow de build Unix classique (configurer, créer, make install) automatise la détection de l'environnement de build et la compilation/installation du logiciel à partir de la source.

```
$ ./configure
$ make
$ sudo make install
```

- Comprendre ce processus est utile pour installer des logiciels non disponibles sous forme de package prédéfini.

```
$ tar xzf app-1.2.tar.gz
$ cd app-1.2
```

Commandes / outils couverts dans ce chapitre

`gcc`	`make`	`./configure`

★ Nouveaux dans ce chapitre : gcc, make, ./configure

Partie IV - Écriture de scripts Shell

Chapitre 24 : Écrire votre premier script

Définit les scripts shell comme des fichiers texte de commandes exécutées par le shell et couvre les mécanismes d'écriture d'un tel script : la ligne shebang, la définition des autorisations exécutables, l'endroit où placer les scripts et les conventions de formatage pour plus de lisibilité.

Concepts et exemples clés

- Le shebang (#!/bin/bash) indique au système quel interprète doit exécuter le script.

```
#!/bin/bash
echo "Hello, World!"
```

- chmod +x est requis avant qu'un script puisse être exécuté directement.

```
$ chmod +x hello.sh
$ ./hello.sh
```

- Un bon formatage (indentation, continuation de ligne avec \) est mis en avant dès le premier script.

```
ls -l \
/etc
```

Commandes / outils couverts dans ce chapitre

`chmod +x`	`#!/bin/bash` (shebang)

Chapitre 25 : Démarrer un projet

Commence un exemple de projet en cours d'exécution : un script minimal de génération de documents, puis l'ajout de données, l'introduction de variables et de constantes (y compris en lecture seule), et ici des documents (<<EOF) pour intégrer du texte multiligne directement dans un script.

Concepts et exemples clés

- Les variables stockent et réutilisent les valeurs tout au long d'un script ; les constantes (lecture seule) empêchent toute modification accidentelle.

```
TITLE="System Report"
readonly TITLE
```

- Ici, les documents fournissent un moyen propre d'incorporer du texte littéral multiligne sans de nombreux appels d'écho distincts.

```
cat <<EOF
Report generated on $(date)
EOF
```

- Le projet construit ici est étendu dans presque tous les chapitres suivants de la partie IV.

```
#!/bin/bash
TITLE="System Report"
echo "$TITLE"
```

Commandes / outils couverts dans ce chapitre

`readonly`	`here documents` (<<)

★ Nouveaux dans ce chapitre : readonly

Chapitre 26 : Conception descendante

Présente les fonctions shell comme moyen de modulariser les scripts à l'aide de principes de conception descendante, couvre les variables locales (limitées à une fonction) et les techniques permettant de garantir que les scripts à exécution longue s'exécutent de manière robuste.

Concepts et exemples clés

- Les fonctions Shell permettent de décomposer un script en blocs nommés et réutilisables.

```
print_header () {
echo "=== $1 ==="
}
print_header "Report"
```

- local restreint la portée d'une variable à la fonction dans laquelle elle est déclarée.

```
count_files () {
local n=$(ls | wc -l)
echo $n
}
```

- Des techniques telles que le piégeage des signaux aident les longs scripts à survivre aux interruptions inattendues.

```
trap 'echo Interrupted; exit' SIGINT
```

Commandes / outils couverts dans ce chapitre

function / local

★ Nouveaux dans ce chapitre : local

Chapitre 27 : Contrôle de flux : branchement avec if

Couvre l'instruction if et l'état de sortie, la commande test pour évaluer les expressions de fichier, de chaîne et d'entier, la syntaxe moderne [[ ]], (( )) pour les entiers et && / || opérateurs de contrôle.

Concepts et exemples clés

- Chaque commande renvoie un état de sortie (0 = succès) sur lequel se branche si/pendant/jusqu'à.

```
grep -q root /etc/passwd
echo $?
```

- test/[ ]/[[ ]] évalue les conditions sur les fichiers, les chaînes et les entiers.

```
if [ -f "$1" ]; then
echo "File exists"
fi
```

- && et || fournir un moyen compact d'enchaîner l'exécution conditionnelle.

```
mkdir backup && cp *.txt backup/ || echo "Failed"
```

Commandes / outils couverts dans ce chapitre

test	[ ]	[[ ]]	(( ))

★ Nouveaux dans ce chapitre : test/[ ], [[ ]], (( ))

Chapitre 28 : Lecture de la saisie au clavier

Couvre la lecture intégrée pour capturer les entrées de l'utilisateur, ses options et la variable IFS, valider les entrées et créer des menus textuels simples.

Concepts et exemples clés

- read est le noyau intégré des scripts interactifs qui demandent à l'utilisateur des valeurs.

```
read -p "Enter your name: " name
echo "Hello, $name"
```

- IFS détermine comment la lecture divise l'entrée en plusieurs variables/champs.

```
IFS=: read user pass uid gid rest < /etc/passwd
```

- La combinaison de read avec case/if crée de simples scripts interactifs pilotés par menu.

```
read -p "Choice (1-3): " choice
case $choice in
1) echo one;;
esac
```

Commandes / outils couverts dans ce chapitre

read

★ Nouveaux dans ce chapitre : read

Chapitre 29 : Contrôle de flux : boucle avec while/jusqu'à

Couvre les boucles while et jusqu'à, la rupture précoce des boucles et le modèle courant de lecture de fichiers ligne par ligne avec une boucle de lecture while.

Concepts et exemples clés

- while et jusqu'à ce que soient des opposés logiques, choisis en fonction de la condition qui se lit le plus naturellement.

```
count=1
while [ $count -le 5 ]; do
echo $count
count=$((count+1))
done
```

- interrompre et poursuivre l'exécution de la boucle de contrôle de l'intérieur.

```
for f in *; do
[ -d "$f" ] && continue
echo "$f"
done
```

- tandis que read line est le modèle de shell idiomatique permettant de traiter un fichier ligne par ligne.

```
while read -r line; do
echo "Line: $line"
done < file.txt
```

Commandes / outils couverts dans ce chapitre

`while`	`until`	`break/continue`

★ Nouveaux dans ce chapitre : while, until

Chapitre 30 : Dépannage

Couvre les catégories d'erreurs de script - syntaxiques et logiques - puis teste avec des cas de test délibérés et des techniques de débogage : traçage de l'exécution et examen des valeurs des variables.

Concepts et exemples clés

- La plupart des bugs de script pour débutants appartiennent à des catégories prévisibles : les guillemets manquants et les fractionnements/globbing inattendus de mots sont particulièrement courants.

```
# Wrong: if [ $var = foo ] (fails if $var is empty)
# Right: if [ "$var" = "foo" ]
```

- Le traçage intégré de bash (set -x) affiche chaque commande au fur et à mesure de son exécution, ce qui est inestimable pour le débogage.

```
$ bash -x myscript.sh
```

- La programmation défensive (vérification des entrées, citation des variables) évite de nombreux bugs avant qu'ils ne surviennent.

```
if [ -z "$1" ]; then
echo "Usage: $0 filename"
exit 1
fi
```

Commandes / outils couverts dans ce chapitre

`set -x (tracing)`	`bash -n (syntax check)`

★ Nouveaux dans ce chapitre : set -x

Chapitre 31 : Contrôle de flux : branchement avec cas

Couvre la commande case pour le branchement multidirectionnel basé sur un modèle, sa syntaxe de modèle et l'exécution de plusieurs actions pour un seul modèle correspondant.

Concepts et exemples clés

- Le cas est idéal pour faire correspondre une valeur unique à plusieurs modèles possibles.

```
case $1 in
start) echo starting;;
stop) echo stopping;;
*) echo "unknown";;
esac
```

- Les modèles prennent en charge les caractères génériques et le | opérateur pour faire correspondre plusieurs alternatives dans une seule clause.

```
case $file in
*.jpg|*.png) echo image;;
*.txt) echo text;;
esac
```

- case est généralement plus lisible qu'une chaîne équivalente d'instructions if/elif/else.

```
case $(uname) in
Linux) echo "Linux system";;
Darwin) echo "macOS";;
esac
```

Commandes / outils couverts dans ce chapitre

```
case
```

★ Nouveaux dans ce chapitre : case

Chapitre 32 : Paramètres de position

Couvre l'accès aux arguments de ligne de commande via les paramètres de position ($1, $2, ...), $#, shift, l'utilisation des paramètres de position dans les fonctions et $@ / $*.

Concepts et exemples clés

- Les paramètres positionnels permettent aux scripts d'accepter des arguments de ligne de commande, tout comme les commandes intégrées.

```
#!/bin/bash
echo "First arg: $1, total args: $#"
```

- shift supprime à plusieurs reprises $1 et renumérote le reste, permettant des boucles sur un nombre arbitraire d'arguments.

```
while [ $# -gt 0 ]; do
echo "$1"
shift
done
```

- "$@" (cité) est l'idiome standard pour transmettre en toute sécurité tous les arguments à une autre commande.

```
log_all () { echo "$@" >> log.txt; }
log_all "$@"
```

Commandes / outils couverts dans ce chapitre

`shift`	`$@/$*/$#`

★ Nouveaux dans ce chapitre : shift

Chapitre 33 : Contrôle de flux : boucle avec for

Couvre le shell traditionnel pour la boucle et la syntaxe de boucle for de style langage C pour l'itération numérique.

Concepts et exemples clés

- Le mot traditionnel sous forme de liste parcourt toute liste séparée par des espaces.

```
for name in Alice Bob Carol; do
echo "Hello, $name"
done
```

- La forme de style C pour ((...)) est préférée pour les boucles numériques classiques basées sur un compteur.

```
for ((i=0; i<5; i++)); do
echo "$i"
done
```

- Le choix du bon formulaire dépend du fait que la boucle parcourt des données ou une plage numérique.

```
for f in *.txt; do
mv "$f" "${f%.txt}.bak"
done
```

Commandes / outils couverts dans ce chapitre

`for`

★ Nouveaux dans ce chapitre : for

Chapitre 34 : Chaînes et nombres

Une plongée approfondie dans l'expansion des paramètres pour la manipulation de chaînes, l'évaluation et l'expansion arithmétiques, les bases numériques, les opérations sur les bits et bc pour les opérations que l'arithmétique entière de bash ne peut pas gérer.

Concepts et exemples clés

- L'expansion des paramètres prend en charge de puissantes opérations de chaîne en ligne sans avoir besoin d'outils externes.

```
file="report.txt"
echo "${file%.txt}" # report
echo "${file/report/summary}"
```

- L'arithmétique Bash concerne uniquement les nombres entiers ; bc est requis pour les calculs à virgule flottante ou de très grande taille.

```
$ echo "scale=2; 10/3" | bc
3.33
```

- La maîtrise de ces extensions permet aux scripts d'éviter de générer des processus externes pour des tâches simples.

```
x=5
echo $((x * 2 + 1)) # 11
```

Commandes / outils couverts dans ce chapitre

`bc`	`$(( ))` (arithmetic expansion)

★ Nouveaux dans ce chapitre : bc

Chapitre 35 : Tableaux

Couvre les tableaux bash : création et attribution de valeurs, accès aux éléments et aux tranches, opérations sur les tableaux et tableaux associatifs.

Concepts et exemples clés

- Les tableaux indexés stockent des listes ordonnées de valeurs sous un seul nom de variable.

```
fruits=(apple banana cherry)
echo "${fruits[1]}" # banana
```

- Les tableaux associatifs (declare -A) fournissent un stockage clé-valeur, similaire aux dictionnaires dans d'autres langages.

```
declare -A colors
colors[apple]=red
echo "${colors[apple]}"
```

- Les opérations sur les tableaux (${#arr[@]}, ${!arr[@]}, arr+=(...)) sont essentielles pour les scripts gérant des collections de données.

```
fruits+=(date)
echo "${#fruits[@]}" # 4
```

Commandes / outils couverts dans ce chapitre

```
declare -a /
declare -A
(arrays)
```

★ Nouveaux dans ce chapitre : declare -a/-A

Chapitre 36 : Exotique

Un ensemble de fonctionnalités avancées du shell : commandes de groupe et sous-shells (y compris la substitution de processus), capture de signal avec trap, exécution asynchrone avec attente et canaux nommés (FIFO).

Concepts et exemples clés

- trap permet à un script d'attraper des signaux (par exemple, SIGINT/Ctrl-C) et d'exécuter du code de nettoyage avant de quitter.

```
trap 'rm -f /tmp/lockfile; exit' SIGINT SIGTERM
```

- wait synchronise un script avec les tâches en arrière-plan qu'il a lancées, utile pour un parallélisme simple.

```
sleep 5 &
sleep 3 &
wait
echo "All background jobs done"
```

- Les canaux nommés (mkfifo) permettent à deux processus autrement indépendants de communiquer via un canal visible par le système de fichiers.

```
$ mkfifo mypipe
$ echo hello > mypipe &
$ cat < mypipe
```

Commandes / outils couverts dans ce chapitre

`trap`	`wait`	`mkfifo`

★ Nouveaux dans ce chapitre : trap, wait, mkfifo

Index principal des commandes (par première apparition)

Ce livre enseigne des dizaines de commandes de manière cumulative, en vue d'un script shell complet. Le tableau ci-dessous regroupe chaque commande/utilitaire majeur par chapitre dans lequel il est introduit pour la première fois, organisé par catégorie fonctionnelle.

Catégorie	Ch.	Commandes
Navigation et exploration de fichiers	2–4	pwd, cd, ls, file, less, mkdir, cp, mv, rm, ln
Aide aux commandes et identification	5	type, which, help, man, apropos, whatis, info, alias
Redirection et filtres	6	cat, uniq, wc, grep, head, tail, tee
Interaction avec le shell	7–8	echo, history (expansion, completion, readline editing)
Autorisations et identité	9	chmod, umask, su, sudo, chown, chgrp, passwd
Processus	10	ps, top, jobs, bg, fg, kill, killall, shutdown
Environnement et éditeurs	11–13	env/printenv, export, source, vi/vim, PS1 prompt customization
Gestion des colis	14	apt-get/apt, dpkg, yum/dnf, rpm
Supports de stockage	15	mount, umount, fdisk, mkfs, fsck, dd
Réseautage	16	ping, traceroute, ip, netstat, ftp, lftp, wget, ssh, scp, sftp
Recherche	17	locate, find, xargs
Archivage et synchronisation	18	gzip, bzip2, tar, zip/unzip, rsync
Regex et traitement de texte	19–20	grep/egrep, sort, cut, paste, join, comm, diff, patch, tr, sed, aspell
Formatage et impression des sorties	21–22	nl, fold, fmt, pr, printf, groff, lpr, lp, a2ps, lpstat, lpq, lprm/cancel
Compilation	23	gcc, make, ./configure
Constructions de script Shell	24–36	test/[ ]/[[ ]], (( )), read, while/until, for, case, shift, bc, declare -a/

Notes finales

La ligne de commande Linux est explicitement une référence d'apprentissage par la pratique : presque chaque chapitre associe des explications à des exercices pratiques, et le projet de script en cours de la partie IV relie presque tous les concepts des parties I à III dans de véritables scripts shell fonctionnels. Les exemples concrets ajoutés tout au long de ce résumé reflètent cet esprit pratique : essayez-les dans un terminal pour renforcer chaque concept.

La valeur fondamentale du livre est cumulative : les chapitres suivants supposent la maîtrise des commandes précédentes (grep, sed, find, redirection, pipelines), et les chapitres sur les scripts de la partie IV combinent presque tout ce qui a été enseigné précédemment dans une automatisation réutilisable. Pour un lecteur qui suit le livre du début à la fin, l'ordre des chapitres de ce résumé reflète le parcours d'apprentissage prévu.